Creating a 2 bit adder based on an FPGA

Levi Terry

UAT

RBT131: Digital Logic

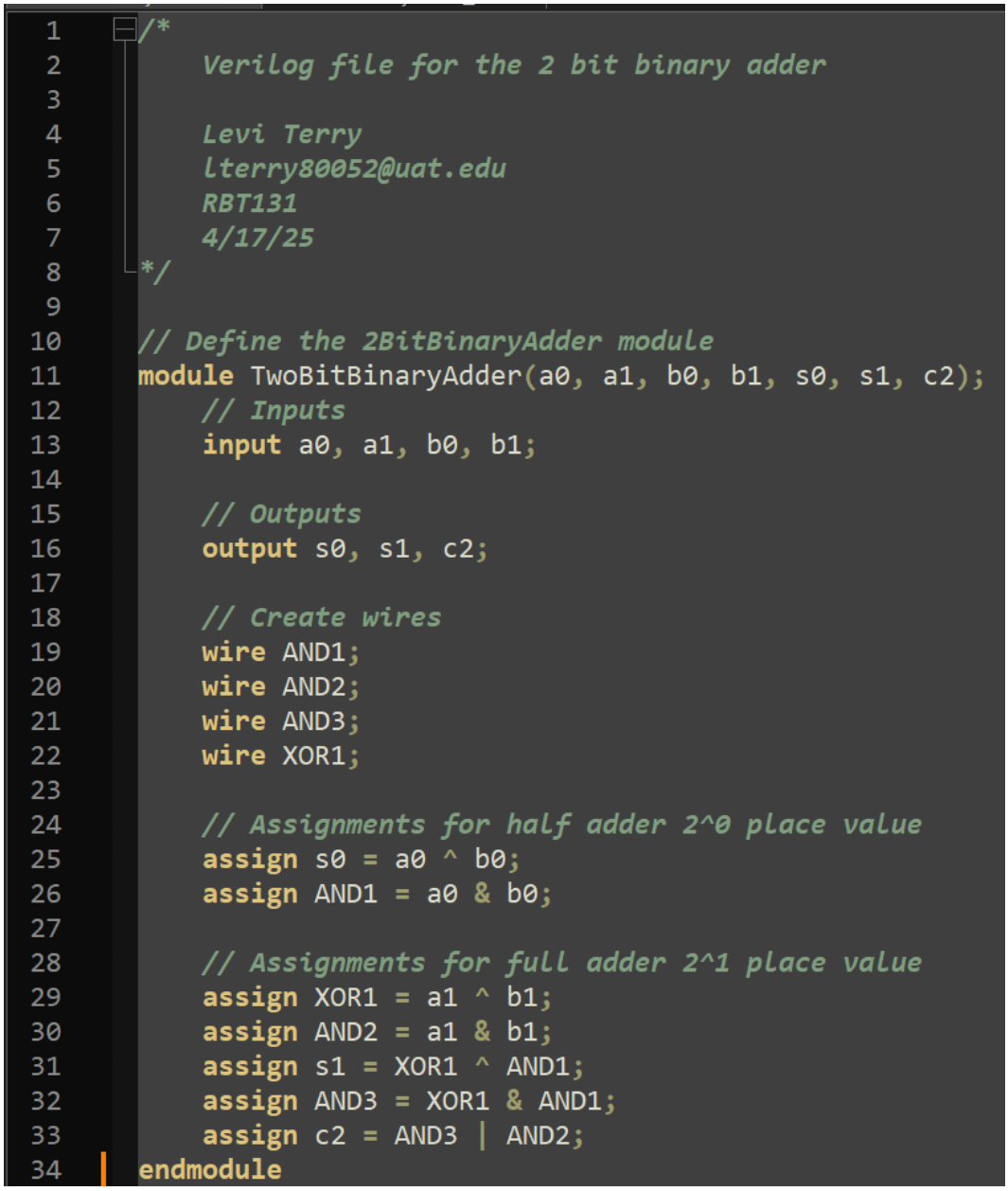
Prof. Prater

5/4/25

Intro

We are attempting to create a two-bit adder for an FGPA. The adder takes in 2 2-bit binary numbers and adds them together (hence the name 2-bit adder). Since there are 4 inputs that can be changed, there are 2^4 = 16 possibilities for the adder. The adder uses both a half adder and a full adder to run its calculations, a0 and b0 run through the half adder, then a1 and b1 along with the carry from the half adder are ran through the full adder to get the final result.

TwoBitBinaryAdder.v



```
/*
    Verilog file for the 2 bit binary adder

    Levi Terry
    lterry80052@uat.edu
    RBT131
    4/17/25
*/

// Define the 2BitBinaryAdder module
module TwoBitBinaryAdder(a0, a1, b0, b1, s0, s1, c2);
    // Inputs
    input a0, a1, b0, b1;

    // Outputs
    output s0, s1, c2;

    // Create wires
    wire AND1;
    wire AND2;
    wire AND3;
    wire XOR1;

    // Assignments for half adder 2^0 place value
    assign s0 = a0 ^ b0;
    assign AND1 = a0 & b0;

    // Assignments for full adder 2^1 place value
    assign XOR1 = a1 ^ b1;
    assign AND2 = a1 & b1;
    assign s1 = XOR1 ^ AND1;
    assign AND3 = XOR1 & AND1;
    assign c2 = AND3 | AND2;
endmodule
```

TwoBitBinaryAdder_tb.v

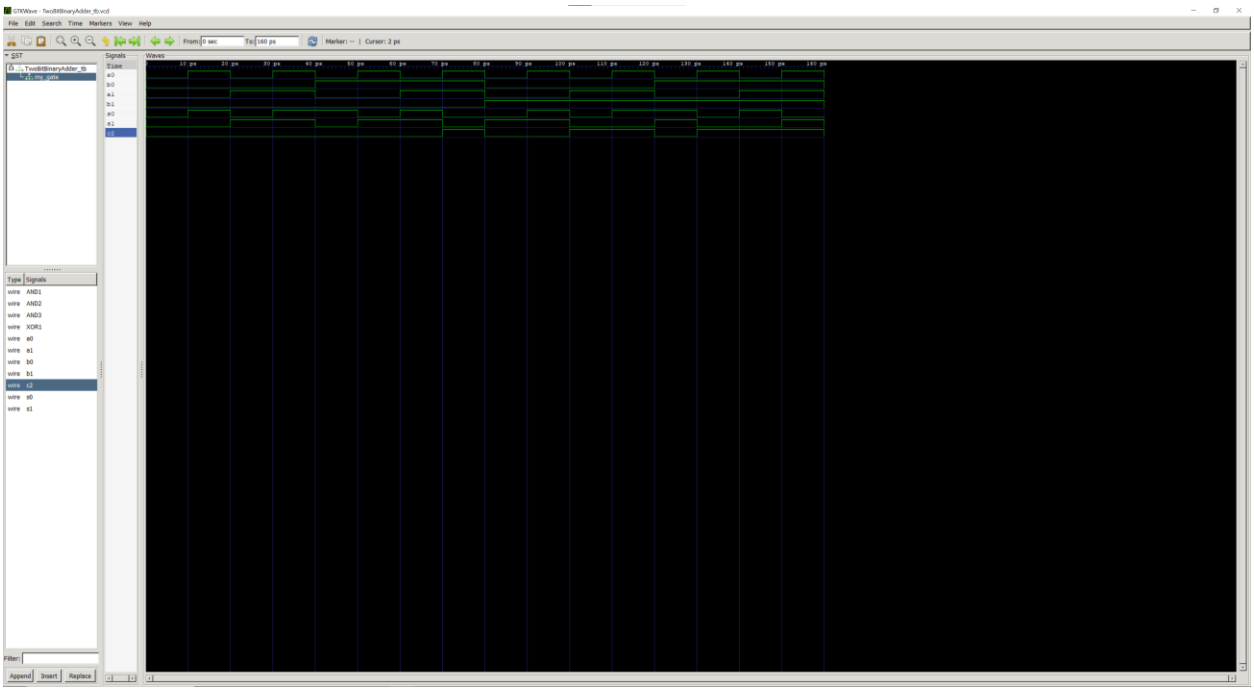
```
/*
    Verilog file for the 2 bit binary adder testbench
    # of possibilities = 2^4 = 2*2*2*2 = 16

    Levi Terry
    lterry80052@uat.edu
    RBT131
    4/22/25
*/

module TwoBitBinaryAdder_tb;
    // Create wires
    wire t_s0, t_s1, t_c2;

    // Create registers
    reg t_a0, t_a1, t_b0, t_b1;

    // How long the sim should run
    localparam DURATION = 160;

    // Create TwoBitBinaryAdder
    TwoBitBinaryAdder my_gate(.a0(t_a0), .a1(t_a1), .b0(t_b0), .b1(t_b1), .s0(t_s0), .s1(t_s1), .c2(t_s2));

    /*
        b: 1 0  | a: 1 0
        ---------------
          0 0       0 0
          0 0       0 1
          0 0       1 0
          0 0       1 1
          0 1       0 0
          0 1       0 1
          0 1       1 0
          0 1       1 1
          1 0       0 0
          1 0       0 1
          1 0       1 0
          1 0       1 1
          1 1       0 0
          1 1       0 1
          1 1       0 0
          1 1       0 1
    */

    initial begin
        // Monitor all wires/registers
        $monitor(t_a0, t_a1, t_b0, t_b1, t_s0, t_s1, t_s2);

        // Set a to 00
        t_a0 = 1'b0;
        t_a1 = 1'b0;
        // Set b to 00
        t_b0 = 1'b0;
        t_b1 = 1'b0;

        // Wait 5 ps
        #10

        // Set a to 01
        t_a0 = 1'b1;
        t_a1 = 1'b0;
        // Set b to 00
        t_b0 = 1'b0;
        t_b1 = 1'b0;

        // Wait 10 ps
        #10

```

```
        // Set a to 00
        t_a0 = 1'b0;
        t_a1 = 1'b1;
        // Set b to 00
        t_b0 = 1'b0;
        t_b1 = 1'b0;

        // Wait 10 ps
        #10

        // Set a to 00
        t_a0 = 1'b1;
        t_a1 = 1'b1;
        // Set b to 00
        t_b0 = 1'b0;
        t_b1 = 1'b0;

        // Wait 10 ps
        #10

        // Set a to 00
        t_a0 = 1'b0;
        t_a1 = 1'b0;
        // Set b to 00
        t_b0 = 1'b1;
        t_b1 = 1'b0;

        // Wait 10 ps
        #10

        // Set a to 00
        t_a0 = 1'b1;
        t_a1 = 1'b0;
        // Set b to 00
        t_b0 = 1'b1;
        t_b1 = 1'b0;

        // Wait 10 ps
        #10

        // Set a to 00
        t_a0 = 1'b0;
        t_a1 = 1'b1;
        // Set b to 00
        t_b0 = 1'b1;
        t_b1 = 1'b0;

        // Wait 10 ps
        #10

        // Set a to 00
        t_a0 = 1'b1;
        t_a1 = 1'b1;
        // Set b to 00
        t_b0 = 1'b1;
        t_b1 = 1'b0;

        // Wait 10 ps
        #10

        // Set a to 00
        t_a0 = 1'b0;
        t_a1 = 1'b0;
        // Set b to 00
        t_b0 = 1'b0;
        t_b1 = 1'b1;

        // Wait 10 ps
        #10
```

```
138
139         // Set a to 00
140         t_a0 = 1'b1;
141         t_a1 = 1'b0;
142         // Set b to 00
143         t_b0 = 1'b0;
144         t_b1 = 1'b1;
145
146         // Wait 10 ps
147         #10
148
149         // Set a to 00
150         t_a0 = 1'b0;
151         t_a1 = 1'b1;
152         // Set b to 00
153         t_b0 = 1'b0;
154         t_b1 = 1'b1;
155
156         // Wait 10 ps
157         #10
158
159         // Set a to 00
160         t_a0 = 1'b1;
161         t_a1 = 1'b1;
162         // Set b to 00
163         t_b0 = 1'b0;
164         t_b1 = 1'b1;
165
166         // Wait 10 ps
167         #10
168
169         // Set a to 00
170         t_a0 = 1'b0;
171         t_a1 = 1'b0;
172         // Set b to 00
173         t_b0 = 1'b1;
174         t_b1 = 1'b1;
175
176         // Wait 10 ps
177         #10
178
179         // Set a to 00
180         t_a0 = 1'b1;
181         t_a1 = 1'b0;
182         // Set b to 00
183         t_b0 = 1'b1;
184         t_b1 = 1'b1;
185
186         // Wait 10 ps
187         #10
188
189         // Set a to 00
190         t_a0 = 1'b0;
191         t_a1 = 1'b1;
192         // Set b to 00
193         t_b0 = 1'b1;
194         t_b1 = 1'b1;
195
196         // Wait 10 ps
197         #10
198
199         // Set a to 00
200         t_a0 = 1'b1;
201         t_a1 = 1'b1;
202         // Set b to 00
203         t_b0 = 1'b1;
204         t_b1 = 1'b1;
205
206         // Wait 10 ps
```

```
207         #10
208
209         // Set a to 00
210         t_a0 = 1'b0;
211         t_a1 = 1'b0;
212         // Set b to 00
213         t_b0 = 1'b0;
214         t_b1 = 1'b0;
215     end
216
217     initial begin
218         // Creat dumpfile and dump vars
219         $dumpfile("TwoBitBinaryAdder_tb.vcd");
220         $dumpvars(0, TwoBitBinaryAdder_tb);
221
222         // Wait for duration
223         #(DURATION);
224     end
225
226 endmodule
```

GTKWave



Questions

1. **How many possibilities do you need for a 2-bit adder?**

   For a 2-bit adder, there are 16 possibilities since 2^4 = 16.

2. **Show you math work for determining simulation length.**

Since there are 16 possibilities and each should run for 10ps, then the simulation should run for 160ps, since 16*10 = 160